

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



Scientific-Analytical Teyf Group

Linux and Python Programming for Data Science and Machine Learning with Telecom and IoT Examples

Seyyed Hossein Hosseini

November 2024

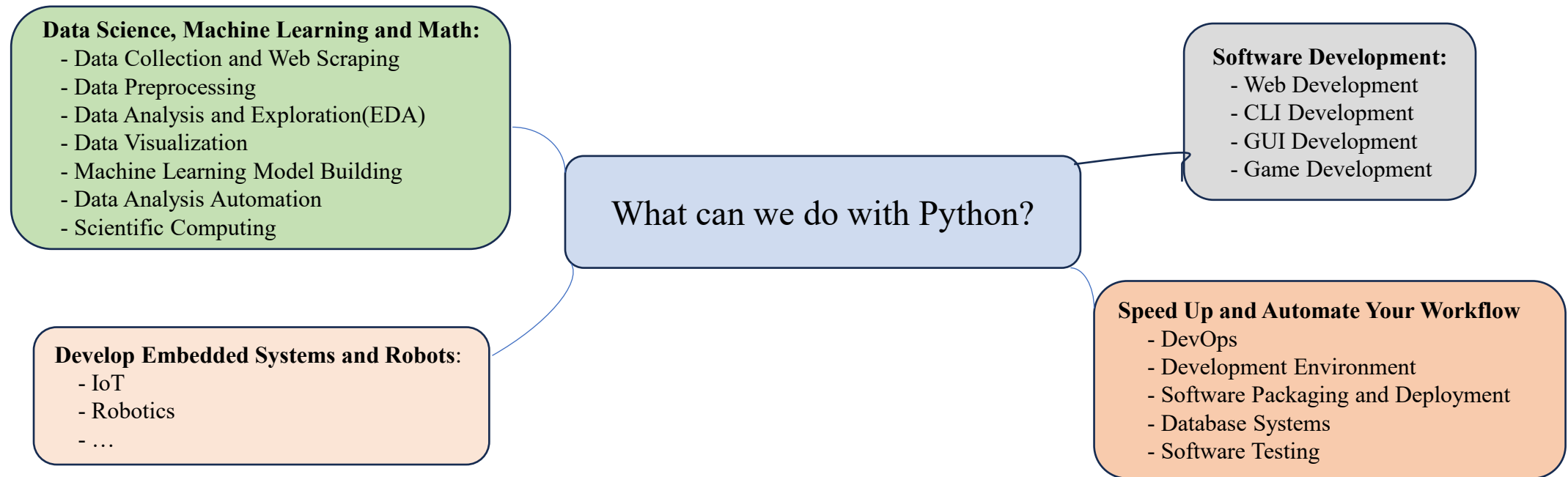
Chapter 4:

Linux and Python Programming for Data Science and Machine Learning with Telecom and IoT Examples

Outline for Part 1	{ <i>An Introduction to Linux Basics</i> <i>What is Programming?</i> <i>Python Basics</i>
Outline for Part 2	{ <i>Conditionals and Loops</i> <i>Functions and Modules</i> <i>Input and Output in Python</i> <i>Object Oriented Programming(OOP)</i>
Outline for Part 3	{ <i>Numpy Library(Numerical Computations)</i> <i>Matplotlib Library(Data Visualization)</i> <i>Pandas Library(Data Analysis)</i>

➤ Python

- **Python** is designed to be a **general-purpose** programming language. You can use Python to do **text analysis**, create **algorithm**, **website**, **GUI**, **game**, and **mobile app**, **automate** repetitive tasks, develop **IoT** solutions and myriad other applications.



- Python is widely used in **data science** and **machine learning**; for these applications, Python is basically just a **calculator**. It's an extremely powerful and versatile calculator.

Python

- Python is currently the most commonly used numerical processing program for data science and machine learning (other contenders include R, MATLAB, Julia, JavaScript, SQL, and C).
- **Question:** Will Python remain the dominant language for data science?
- Anyway, the good news is that programming languages have strong **transfer learning**, which means that building Python proficiency will help you learn other languages. In other words, time spent learning Python is invested, not wasted.

❖ IDEs (Interactive Development Environments)

- Python is the programming language, and you can run Python in many different applications, which are called **environments**. Different environments are created by different developers with different preferences and needs.
- Some common IDEs that you might encounter include **Visual Studio Code**, **Spyder**, and **PyCharm**. Perhaps the most commonly used IDE for learning Python is called **Jupyter notebooks**.
- The Jupyter notebook is an **open source web application** that allows you to create and share documents that contain live code, visualizations and some explanatory text as well.

Python

❖ Using Python Locally and Online

- Because Python is **free** and **lightweight**, it can be run on a variety of **operating systems**, either on **your computer** or in a **cloud server**:

❑ Running Python locally

- You can install Python on any major operating system (Windows, Mac, Linux). If you are comfortable with installing programs and packages, then you can install libraries as you need them.

❑ Running Python online

- While going through this course, I recommend running Python on the web. Advantages of **cloud-located Python** include that **you don't need to install anything locally**, **you don't need to use your own computing resources**, and **you can access your code from any browser on any computer and any operating system**.
- We use **Google's Colaboratory environment** because it syncs with **Google Drive**. That allows to keep Python code files on Google Drive and then open them from <https://colab.research.google.com>. There are several other **cloud-based Python environments** that you can use (**AWS**, **GCP**, **Microsoft Azure**, and **GitHub Codespaces**)
- Google Colab is **free** to use. You'll need a Google account to access it. Then you can simply upload the code files to your Google Drive and open them in Colab.

Python

❖ Libraries

- Python is intentionally designed for quick and easy installation and execution. However, the trade-off is that the **standard Python installation** includes a limited set of **built-in** functions. As a result, developers create collections of functions focused on specific topics, known as **libraries/packages**.
- Once a library/package is **imported** into Python, all of its **variables**, **functions**, and **classes** become accessible.
- A Google search indicates that there are over 130,000 libraries available for Python, but there's no need to memorize them all.
- We will primarily utilize a select **few libraries/packages** tailored for **numerical processing** and data **visualization**. The most essential library for linear algebra is known as **NumPy**, a blend of "numerical Python".
- **Python libraries/packages are distinct from the core Python installation**, requiring separate downloads from the web, after which they can be **imported** into Python for use. While they only need to be downloaded once, they must be **re-imported** into Python in each session.
- If you have installed Python using **Anaconda** or if you are working in **Google's Colab environment**, there is no requirement to download any libraries/packages for this course. However, you will still need to import them as necessary.

Python

❖ Difference Between Script, Function, Module, Package, and Library in Python

▪ Script:

- A script refers to a Python file designed to be executed directly. Upon execution, it is expected to perform a specific action. As a result, scripts frequently contain code that is not encapsulated within any classes or functions.

▪ Function:

- A function in Python is a **block of organized, reusable code** that is used to perform a single, related action. It provides better **modularity** for your application and a high degree of code **reusability**.
- Functions are defined using the *def* keyword and can take parameters and return values.
- Examples of **built-in** Python functions include `print()`, `len()`, and **user-defined** functions like `calculate_speed()`.

▪ Module:

- A module is a simple Python file that contains collections of **functions, classes**, and **global variables**, typically with a **.py** extension.
- It is an executable file and serves to organize **related code** into a single file for better **maintainability** and **reusability**.
- A module is intended to be **imported** into **scripts** or other modules.

Python

❖ Difference Between Script, Function, Module, Package, and Library in Python

▪ Package:

- A package in Python is a collection of **related modules** that work together to provide certain functionality.
- It is essentially a **directory** containing Python **modules** and an **`__init__.py`** file to indicate to the Python **interpreter** that it should be processed as a package.
- Packages allow for a hierarchical organization of modules and provide a way to manage and distribute Python projects effectively.
- Packages can contain **subpackages** and they promote code **reusability** and **maintainability** by grouping related functionality together.

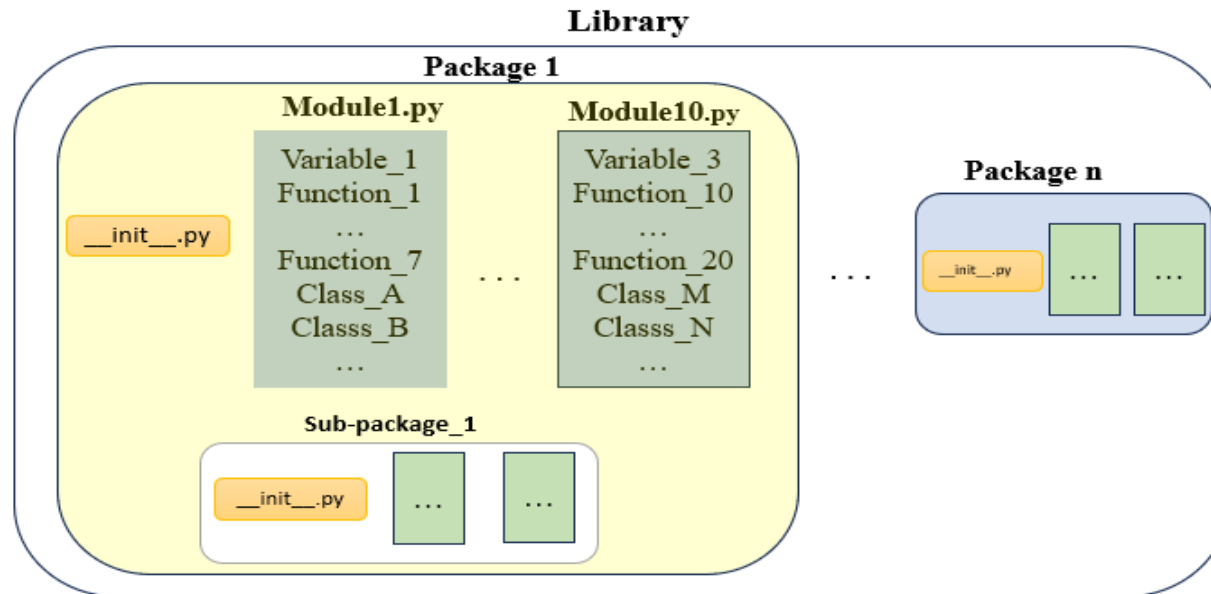
Python

❖ Difference Between Script, Function, Module, Package, and Library in Python

▪ Library:

- A library in Python is a **collection of packages**. It is an umbrella term that means a bundle of code, often containing multiple packages and modules.
- Libraries can be built to share **reusable code** with the community, providing a set of useful functions related to the same area to **eliminate the need for writing code from scratch**.

➤ Summary



Python

- **Introduction to Google Colab**
- **Working with Code Files in Google Colab**

Recommended Textbooks

[1] “Practical Linear Algebra for Data Science, From Core Concepts to Applications Using Python” by Mike X Cohen

[2] “Introduction to Programming in Python, An Interdisciplinary Approach” by Robert Sedgewick, Kevin Wayne and Robert Dondero